The Chain Rule

Problem: Can we use Conditional complexity to compute complexity step-by-step?

Conditional complexity can be used to capitalize on what an agent (represented as a machine) may already know. It is also a great tool to compute estimates of description complexity, step-by-step, using the **chain rule**.

If we already know an object (think for instance of a number) s_2, conditional complexity allows us to compute the complexity $C(s_1|s_2)$ of another object s_1 knowing s_2. We would like to say that the complexity $C(s_1)$ of s_1 is simply the complexity $C(s_2)$ of s_2 plus $C(s_1|s_2)$. This is almost what we have:

Chain rule 1 for "unambiguous" program codes

$$C(s_1) \leq C(s_2) + C(s_1 \mid s_2) + O(1).$$

Here $O(1)$ designates some constant.
This is almost what we wanted! There is an inequality instead of an equality. It comes from the fact that the detour through s_2 might not be optimal.

Caveat: The chain rule is true only if the machine can delimit programs (*i.e.* it can tell where the program that generates s_2 ends and the program that generates s_1 from s_2 begins). This property is guaranteed if program codes are "unambiguous" (or uniquely decodable). A formal definition of this property will be given in Chapter 3.

▶ Proof of the chain rule

Chain Rule

0/1 point (ungraded)

Using the coding program `PositionalCoding.py`, compute C(m) + C(n | m) for n = 2811 and m = 2700 (do not count space separators).

What do you get?

- ○ 7 bits
- ◉ 11 bits
- ○ 17 bits ✔
- ○ 18 bits
- ○ 21 bits

✖

Answer
Incorrect:
The code for m = 2700 is `0 1101 00` and the conditional code for n = 2811 knowing m is `100 1 110001`. So the correct answer is: 7 + 10 = 17 (remember that we do not count space separators here). Note that in this case, $C(n)$ is strictly smaller than $C(m) + C(n \mid m)$.

Submit **You have used 1 of 1 attempt**

Answers are displayed within the problem

The chain rule can also been written as follows:

Chain rule 2 for "unambiguous" program codes

Chain rule 2 for "unambiguous" program codes

$$C(s_1, s_2) \leq C(s_1) + C(s_2 \mid s_1) + O(1).$$

It comes from the fact that a good way to compute s_1 and s_2 consists of starting with computing one of them, before computing the other one. That "good way" might not be the shortest way, though (hence the inequality).

Chain rule and coincidences

Problem: Are there practical applications of (conditional) complexity?

Algorithmic Information Theory has many practical applications. Some of them will be explored in the following chapters. Let's mention a situation of everyday life in which Conditional complexity plays a central role: Coincidences.

ɛd.

ıt the last draw is
ıy?

ɔomplexity of the last
ɟerring to the

ɾule:

$r_{-1}) + O(1).$

$r_{-1}) + O(1).$

Lottery

0/1 point (ungraded)

The same winning combination 13, 14, 26, 32, 33, 36 was produced by Israel's national biweekly (= twice a week) lottery on September 21st and on October 16th, 2010. Can you quantify the simplicity of the second event by the time it occurred ?

- (●) 1 bit
- ( ) 3 bits ✔
- ( ) 12 bits
- ( ) 30 bits

✖

Answer
Incorrect:
The preceding, identical, draw is located at position $k = 7$ in the list of past rounds.
$C(r_0) \leq C(r_{-k}) + C(r_0 | r_{-k}) + O(1).$

The complexity of the second round, the day it occurred, is $C(r_{-k}) = C(k)$ which is about 3 bits

Submit You have used 1 of 1 attempt

Answers are displayed within the problem

Previous Next